knowledge-supplememt

- 亚声速：(速度 < 声速)，喷管 “收窄加速、扩宽减速”；
- 超声速：(速度 > 声速)，喷管 “扩宽加速、收窄减速”。

要理解超声速流动中 “喷管扩宽加速、收窄减速” 的规律，需突破 **“不可压缩流体” 的直觉经验**，关注**可压缩流体（气体）在超声速下的特殊行为**，核心在于**气体的可压缩性和马赫数的物理意义**：

1. 先明确：声速是 “扰动传播的临界速度”

声速 c 是气体中**压力扰动（如声波）的传播速度**，反映了气体 “抵抗压缩” 的能力。当流体速度 v 超过声速（(Ma>1)）时，流动的 “扰动”（如压力、密度变化）**无法逆流传播**，气体的可压缩性会以和亚声速完全不同的方式体现。

2. 从 “质量守恒” 和 “能量守恒” 分析

对于定常流动，**质量守恒**要求 “单位时间通过任意截面的质量流量相等”，即：(\rho v A = 常数)（(\rho) 是气体密度，v 是流速，A 是喷管截面积）

同时，**能量守恒**（**等熵**流动下）要求 “气体的内能与动能相互转换”：速度增加时，气体密度会**急剧减小**（因为动能增加，内能减少，气体膨胀）。

3. 超声速时的特殊逻辑（(Ma>1)）

当 (v > c)（超声速），如果要让速度**继续增加**，由于动能大幅增加，气体必须**剧烈膨胀**（密度(\rho)急剧减小）。根据质量守恒 (\rho v A = 常数)，(\rho) 减小的幅度远大于 v 增加的幅度，因此**必须增大截面积 A**，才能保证质量流量守恒。

反之，如果喷管**收窄**（A 减小），为了满足质量守恒，要么 v 减小，要么 (\rho) 增大，但超声速下气体 “膨胀趋势” 极强，密度无法大幅增大，因此**速度会被迫减小**。

4. 举个直观例子：火箭喷管

火箭发动机的喷管是**先收窄（喉道）、后扩宽**的形状：

- 燃气在喉道前是亚声速，收窄加速到喉道处达到声速；
- 喉道后是扩张段，燃气在扩张段内**超声速流动且持续加速**，最终以极高速度喷出，产生巨大推力。

这就是 “超声速扩宽加速” 的实际应用，若没有扩张段，超声速燃气的速度会被限制，推力大幅降低。

总结：“反直觉” 的本质是可压缩性的主导作用

亚声速流动中，气体可压缩性弱，近似 “不可压缩”（比如水流），所以 “收窄加速、扩宽减速” 符合直觉；但超声速下，气体可压缩性极强，“膨胀 - 密度剧减” 的效应主导了流动规律，因此呈现 “扩宽加速、收窄减速” 的反直觉现象。

有限差分方法是一种 “把连续的微分方程变成离散的代数方程，再用计算机计算” 的数值工具，常用于像喷管流动这类复杂流体问题的求解。

7.3节算例

气体从大储气罐（压力温度恒定）流入“收缩 - 扩张喷管”，最终以超声速流出，我们要计算喷管内每个位置的密度、速度、温度、压力这些参数，最后和理论精确解对比，验证方法的正确性。

关键规律：

1. 喷管形状：先收缩（截面积变小）再扩张（截面积变大），中间最细的地方叫“喉部”（A/A*=1）。
2. 流动特点：收缩段气体从亚声速（Ma<1）加速，喉部刚好达到声速（Ma=1），扩张段继续加速到超声速（Ma>1）。
3. 核心约束：质量守恒、动量守恒、能量守恒（等熵流动，没有能量损失）。

状态方程是描述**流体的压力、密度、温度三者之间关系**的公式，**理想气体状态方程**：p = ρ RT

p 是压力，ρ 是密度，T 是温度，R 是气体常数（比如空气的 $R \approx 287\,\mathrm{J/(kg{\cdot}K)}$）

编程流程：

步骤 1：初始化参数

先设定所有固定参数，对应书里的假设：

- 气体参数：比热比 γ=1.4（空气的标准值），Courant 数 C=0.5（稳定条件，必须≤1）；
- 喷管参数：喷管长度 L=3.0（无量纲），网格点数 N=31（x 从 0 到 3.0，Δx=0.1）；
- 喷管形状：抛物线型（书里式 7.73），核心是计算每个网格点的截面积比 A/A*（喉部 x=1.5 处 A/A*=1）；
- 初始条件：t=0 时，参数随 x 线性变化（书里式 7.74），比如 ρ'=1-0.3146x，V'=(0.1+1.09x)*sqrt(T')，T'=1-0.023x。

步骤 2：离散控制方程（核心，对应书里 7.3.1）

书里推导了 3 个控制方程（连续、动量、能量）的无量纲离散式，直接翻译成 Python 的数组运算即可，核心是“差分替换导数”：

- 向前差分：f'(x) ≈ [f (x+Δx) - f (x)] / Δx（预测步用）；
- 向后差分：f'(x) ≈ [f (x) - f (x-Δx)] / Δx（修正步用）。

举个例子（连续方程的预测步，书里式 7.51）：

```
# 预测步：计算密度的时间导数（向前差分）
d_rho_dt_pred = -rho[i] * (V[i+1] - V[i])/dx - rho[i]*V[i]*(np.log(A[i+1]) - np.log(A[i]))/dx - V[i]*(rho[i+1] - rho[i])/dx
```

动量方程、能量方程同理，只是公式不同，直接照书里的式 7.52、7.53 翻译，变量都是数组（每个网格点的数值）。

步骤 3：**预测步**（估算 t+Δt 的参数）

1. 对每个内点（不是边界点），用向前差分计算 ρ、V、T 的时间导数（d_rho_dt_pred、d_V_dt_pred、d_T_dt_pred）；
2. 计算时间步长 Δt：按 CFL 条件（书里式 7.67），Δt=C*Δx/(a [i]+V [i])，a 是当地声速（a=sqrt (T [i])，无量纲），取所有网格点 Δt 的最小值（保证稳定）；
3. 估算预测值：**ρ_pred = ρ + d_rho_dt_pred * Δt**，V_pred、T_pred 同理。

步骤 4：**修正步**（修正预测值）

1. 用预测值 ρ_pred、V_pred、T_pred，对每个内点用向后差分计算时间导数（d_rho_dt_corr、d_V_dt_corr、d_T_dt_corr）；
2. 取时间导数的平均值：d_rho_dt_avg = 0.5*(d_rho_dt_pred + d_rho_dt_corr)；
3. 更新最终的 t+Δt 参数：**rho_new = rho + d_rho_dt_avg * Δt**，V_new、T_new 同理。

步骤 5：处理边界条件（容易踩坑）

书里的边界条件要严格遵守：

- 入口（i=0，亚声速）：给定 ρ'=1.0、T'=1.0（储气罐参数），速度 V1 通过内点 V2、V3 外插（V1=2*V2 - V3，书里式 7.70）；
- 出口（i=N-1，超声速）：所有参数（ρ、V、T）都通过内点外插（比如 V_out=2*V [N-2] - V [N-3]，书里式 7.72a）。

步骤 6：迭代到稳态

重复 “步骤 2 - 步骤 5”，直到相邻两次迭代的参数变化小于阈值（比如 1e-6），说明达到稳态。最后计算压力 p=ρ*T（无量纲），马赫数 Ma=V/sqrt (T)，和书里的精确解对比。

无量纲化：

（即物理量除以一个 “参考值”，初始压力 p_0、密度 ρ_0、温度 T_0）：让编程时的热力学参数计算更高效

- 无量纲密度 $\rho' = \rho/\rho_0$
- 无量纲温度 $T' = T/T_0$
- 无量纲压力 $p' = p/p_0$

把这些代入，得到：$p_0 p' = (\rho_0 \rho') \cdot R \cdot (T_0 T')$两边同时除以 p_0，整理后：$p' = \rho' \cdot \left(\frac{\rho_0 R T_0}{p_0}\right) \cdot T'$

近似为1，即：$p'=\rho' \cdot T'$。

比热比：

比热比$\gamma = 1.4$（空气的标准值）

比热比γ是定压比热容c_p与定容比热容c_v 的比值，即：$\gamma = \frac{c_p}{c_v}$

- c_p：单位质量的气体在**压力不变**时，温度升高 1K 所需的热量。
- c_v：单位质量的气体在**体积不变**时，温度升高 1K 所需的热量。

比热比反映了气体在**等熵过程**（无能量损失、可逆的绝热过程）中，温度、压力、密度等状态参数的变化规律。在气体动力学（如喷管流动、超声速流动）中，它是描述气体热力学特性的**核心参数**，直接影响：

- 声速的计算（声速公式$a = \sqrt{\gamma R T}$，其中R是气体常数，T是温度）；
- 等熵过程中压力、密度、温度的关系（如$\frac{p_2}{p_1} = \left(\frac{\rho_2}{\rho_1}\right)^{\gamma} = \left(\frac{T_2}{T_1}\right)^{\frac{\gamma}{\gamma-1}}$）。

控制方程：（有待理解。。。）

一、连续方程（质量守恒）

物理意义：“流体的质量不会凭空产生或消失”。想象水流过喷管，每个位置的 “流入质量” 必须等于 “流出质量 + 控制体内质量的变化”。

- 无量纲化后，方程描述的是：

  「密度随时间的变化」 + 「密度与速度的空间变化」 + 「密度速度随截面积的变化」 = 0

- 离散到编程中（预测步的向前差分）：

  用相邻网格点的

  差值 ÷ 网格步长

  ((f[i+1]-f[i])/dx）来近似 “空间变化率”，进而得到「密度的时间变化率d_rho_dt」。

  对应代码中的式子：

  ```
  d_rho_dt = -rho[i] * (V[i+1] - V[i])/dx - rho[i]*V[i]*(np.log(A[i+1]) - np.log(A[i]))/dx - V[i]*(rho[i+1] - rho[i])/dx
  ```

  每个项的物理含义：

  - `rho[i] * (V[i+1] - V[i])/dx`：速度的空间变化导致的 “质量流走速率”；
  - `rho[i]*V[i]*(np.log(A[i+1]) - np.log(A[i]))/dx`：截面积变化导致的 “质量流量变化速率”；
  - V[i]*(rho[i+1] - rho[i])/dx：密度自身的空间变化导致的 “质量变化速率”；

    这三项的负值就是 “密度随时间的变化率”（因为质量守恒要求这些变化相互平衡）。

二、动量方程（动量守恒）

物理意义：“流体的动量变化由‘力’驱动”。类比开车时，油门 / 刹车（力）会改变车速（动量）；流体中，压力差、密度变化会改变流速。

- 无量纲化后，方程描述的是：

  「速度随时间的变化」 + 「速度的空间变化」 + 「压力 / 密度的空间变化」 = 0

- 离散到编程中（预测步的向前差分）：

  同样用 “相邻网格点的差值 ÷ 步长” 近似空间变化率，得到「速度的时间变化率d_V_dt」。

  对应书里的式 7.52，代码逻辑类似：

  ```
  d_V_dt = -V[i]*(V[i+1]-V[i])/dx - (1/gamma)*((T[i+1]-T[i])/dx + (T[i]/rho[i])*(rho[i+1]-rho[i])/dx)
  ```

  每个项的物理含义：

  - `V[i]*(V[i+1]-V[i])/dx`：速度自身的空间变化（“惯性项”，比如高速流动时的动量传递）；
  - `(1/gamma)*((T[i+1]-T[i])/dx + (T[i]/rho[i])*(rho[i+1]-rho[i])/dx)`

    ：压力梯度项（压力由p=rho*T决定，所以压力变化由温度和密度的空间变化共同驱动）；

    这两项的负值就是 “速度随时间的变化率”（力的作用导致速度变化）。

三、能量方程（能量守恒）

物理意义：“流体的能量（温度）变化由‘机械能与内能的转化’驱动”。比如高速流动时，动能和内能会相互转化（绝热压缩 / 膨胀）。

- 无量纲化后，方程描述的是：

  「温度随时间的变化」 + 「速度导致的温度空间变化」 + 「截面积变化导致的能量转化」 = 0

- 离散到编程中（预测步的向前差分）：

  用 “相邻网格点的差值 ÷ 步长” 近似空间变化率，得到「温度的时间变化率d_T_dt」。

  对应书里的式 7.53，代码逻辑类似：

  ```
  d_T_dt = -V[i]*(T[i+1]-T[i])/dx - (gamma-1)*T[i]*((V[i+1]-V[i])/dx + V[i]*(np.log(A[i+1])-np.log(A[i]))/dx)
  ```

  每个项的物理含义：

  - `V[i]*(T[i+1]-T[i])/dx`：速度导致的温度空间变化（比如流速快的地方温度如何变化）；
  - `(gamma-1)*T[i]*((V[i+1]-V[i])/dx + V[i]*(np.log(A[i+1])-np.log(A[i]))/dx)`

    ：动能与流动功转化为内能的部分（

    gamma-1是热力学系数，反映内能和动能的转化关系）；

    这两项的负值就是 “温度随时间的变化率”（能量转化导致温度变化）。

总结：三个方程的 “编程逻辑共性”

这三个方程都是 **“守恒定律” 的数学表达**，离散时都用 **“相邻网格点的差值 ÷ 步长” 来近似空间变化率**，进而得到 “时间变化率”，最终在预测 - 修正步中更新每个网格点的密度、速度、温度。

你可以把这三个方程想象成 “三个联动的计算器”：

- 连续方程算「密度怎么变」；
- 动量方程算「速度怎么变」；
- 能量方程算「温度怎么变」；

  它们一起迭代，直到喷管内的流场达到稳定（稳态）。

fsolve:

`fsolve` 是 `scipy.optimize` 里的数值求解函数，它的作用是：

- 接收一个待求解的方程（函数）和一个初始猜测值；
- 从初始猜测值出发，迭代找到使方程结果为 0 的未知数。

`lambda` 是用于创建匿名函数（即没有显式函数名的函数）的关键字，语法格式为 `lambda 参数: 表达式`，它会返回一个函数对象，可直接用于需要函数作为参数的场景（比如 `fsolve`）。

作用域：

在 Python 中，`if/else`、`for/while`**等控制流结构并不会创建新的局部作用域**—— 只有函数（`def`）、类（`class`）、模块，以及 Python 3 中的`with`块或`lambda`表达式才会创建新作用域。

print格式补充：

- {'I':>4s}：
  - `'I'`：要输出的列标题文本（这里是第一列标题 “I”）；
  - :>4s：格式说明符，具体含义：
    - `>`：表示**右对齐**（`<`是左对齐，`^`是居中对齐）；
    - `4`：表示该列占用的**字符宽度为 4**（不足时用空格补全，超过时按实际长度显示）；
    - `s`：表示按**字符串类型**格式化（对文本标题来说可省略，默认按字符串处理）。

疑问：

这**两个无量纲声速**，本质是**同一个实际物理声速**，只是无量纲化时 “除以的参考声速不同”：

- 实际物理声速（有量纲）：$a = \sqrt{\gamma R T}$；
- 第一个无量纲声速：$\frac{a}{\sqrt{RT_0}} = \sqrt{\gamma \times \frac{T}{T_0}} = \sqrt{\gamma T}$（因为$\frac{T}{T_0}$是无量纲温度）；
- 第二个无量纲声速：$\frac{a}{\sqrt{\gamma R T_0}} = \sqrt{\frac{T}{T_0}} = \sqrt{T}$。

所以结论是：这两个无量纲声速**数值不同**，但对应**同一个实际物理声速**；代码中用不同形式，是因为精确解（第一种）和数值解（第二种）采用了不同的无量纲化基准，不影响最终 “相对物理关系” 的计算（比如马赫数(Ma=V/a)的比值会自动抵消基准差异）。

辨析：

1. **先明确两个核心定义**

- `V/a₀`中的`a₀`：是**滞止声速**（气罐内的声速），公式为`a₀ = √(γT₀)`（`T₀`是气罐滞止温度）。
- 马赫数`Ma`中的`a`：是**当地声速**（喷管内该位置的声速），公式为`a = √(γT)`（`T`是当地温度）。

2. **两者的数学关系**

马赫数的定义是`Ma = V / a`，结合`a₀`和`a`的公式，可推导出：

$$Ma = \frac{V}{a} = \frac{V}{\sqrt{\gamma T}} = \frac{V}{\sqrt{\gamma T_0}} \times \sqrt{\frac{T_0}{T}} = \left(\frac{V}{a_0}\right) \times \sqrt{\frac{T_0}{T}}$$

```
cpu > inverse_metrix.cpp > main()
1   #include <iostream>
#include errors detected. Please update your includePath. Squiggles are disabled for this translation unit (E:\Myself\postgraduate\C_Cplusplus\VSC\chasing_method\cpu\inverse_metrix.cpp). C/C++(1696)
could not open source file "iostream" (no directories in search list). Please run the 'Select IntelliSense Configuration...' command to locate your system headers. C/C++(1696)
View Problem (Alt+F8)    Quick Fix... (Ctrl+.)
```

这种是缺少C++编译器

MinGW-W64-builds

Installation: GitHub

直接下载编译之后的，特征：有bin

```
(base) PS E:\Myself\postgraduate\C_Cplusplus\VSC\chasing_method>  & 'c:\Users\23608\.vscode\extensions\ms-vscode.cpptools-1.29.3-win32-x64\debugAdapters\bin\WindowsDebugLauncher.exe' '--stdin=Microsoft-MIEngine-In-is432us1.z0y' '--stdout=Microsoft-MIEngine-Out-aglcei2a.0np' '--stderr=Microsoft-MIEngine-Error-r22u5to5.oj1' '--pid=Microsoft-MIEngine-Pid-3jnzorft.kmp' '--dbgExe=E:\ProgrammingSoftware\Env\C++interpreter\mingw\x86_64-15.2.0-release-win32-seh-ucrt-rt_v13-rev0\mingw64\bin\gdb.exe' '--interpreter=mi'
aaa
```

vsc这里多余的输出也不知道是为啥。

学习cuda:

```
#include <iostream>
#include <cuda_runtime.h>

// CUDA 错误检查宏
#define CUDA_CHECK(err) { \
    cudaError_t error = err; \
    if (error != cudaSuccess) { \
        fprintf(stderr, "CUDA Error at %s:%d - %s\n", __FILE__, __LINE__,
cudaGetErrorString(error)); \
        exit(EXIT_FAILURE); \
    } \
}

// 1. 核函数定义
// 在 GPU 上执行向量加法
__global__ void vectorAddKernel(const float *A, const float *B, float *C, int N)
{
    // 2. 线程组织：计算全局线程ID
    int tid = blockIdx.x * blockDim.x + threadIdx.x;

    // 边界检查，防止数组越界
    if (tid < N) {
        C[tid] = A[tid] + B[tid];
    }
}
```

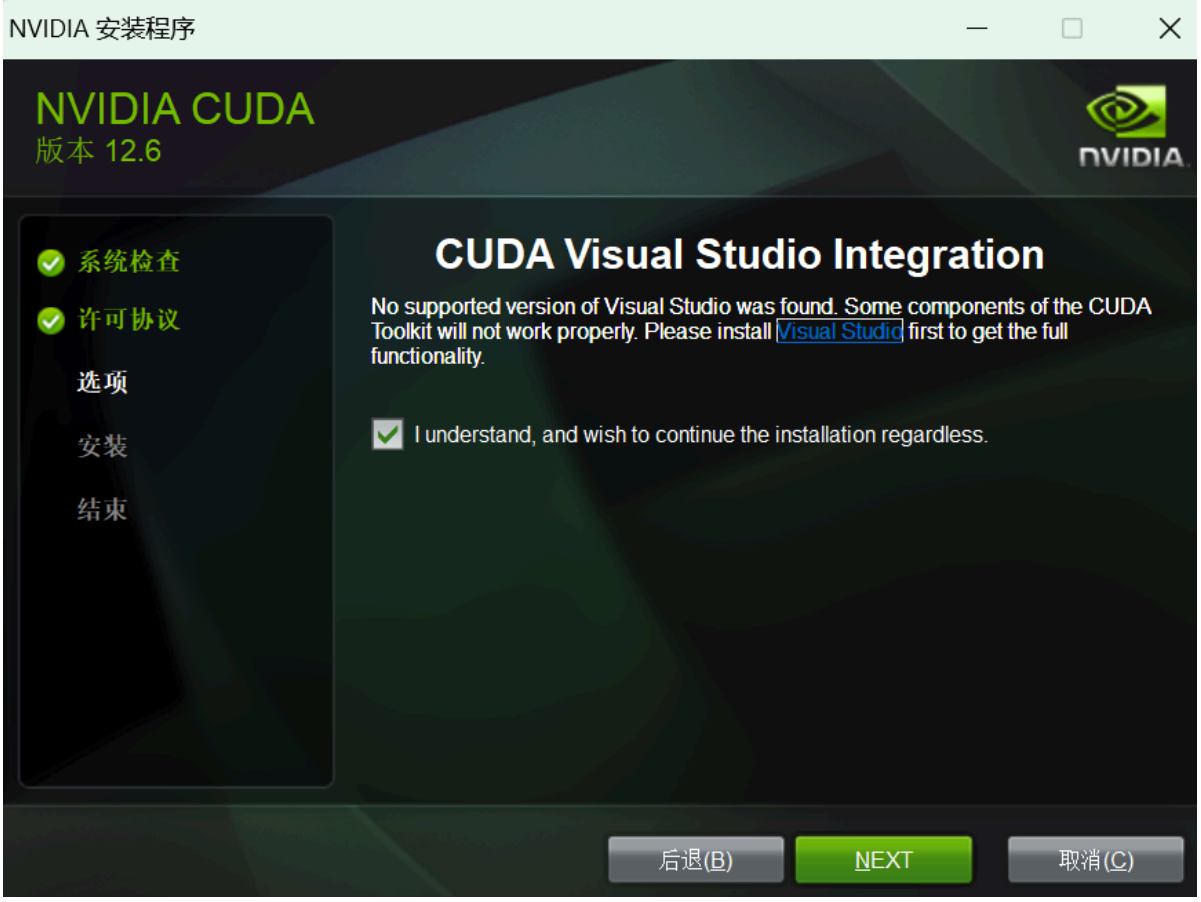
```
int main() {
    int N = 1 << 20; // 1048576 个元素
    size_t size = N * sizeof(float);

    // 在主机上分配内存
    float *h_A = new float[N];
    float *h_B = new float[N];
    float *h_C = new float[N];

    // 初始化主机数据
    for (int i = 0; i < N; ++i) {
        h_A[i] = static_cast<float>(rand()) / RAND_MAX;
        h_B[i] = static_cast<float>(rand()) / RAND_MAX;
    }

    // 3. & 4. GPU 内存管理与数据传输
    float *d_A, *d_B, *d_C;

    // 在设备上分配全局内存
    CUDA_CHECK(cudaMalloc(&d_A, size));
    CUDA_CHECK(cudaMalloc(&d_B, size));
    CUDA_CHECK(cudaMalloc(&d_C, size));

    // 将数据从主机拷贝到设备
    CUDA_CHECK(cudaMemcpy(d_A, h_A, size, cudaMemcpyHostToDevice));
    CUDA_CHECK(cudaMemcpy(d_B, h_B, size, cudaMemcpyHostToDevice));

    // 配置核函数启动参数
    int threadsPerBlock = 256;
    // 计算需要的线程块数量，确保能覆盖所有N个元素
    int blocksPerGrid = (N + threadsPerBlock - 1) / threadsPerBlock;

    // 启动核函数
    vectorAddKernel<<<blocksPerGrid, threadsPerBlock>>>(d_A, d_B, d_C, N);

    // 检查核函数启动是否有错误
    CUDA_CHECK(cudaGetLastError());
    CUDA_CHECK(cudaDeviceSynchronize()); // 等待GPU完成所有任务

    // 将结果从设备拷贝回主机
    CUDA_CHECK(cudaMemcpy(h_C, d_C, size, cudaMemcpyDeviceToHost));

    // 验证结果
    bool success = true;
    for (int i = 0; i < 10; ++i) { // 只验证前10个
        if (abs(h_C[i] - (h_A[i] + h_B[i])) > 1e-5) {
            success = false;
            break;
        }
    }
    if (success) {
        std::cout << "Vector addition successful!" << std::endl;
        std::cout << "A[0] = " << h_A[0] << ", B[0] = " << h_B[0] << ", C[0] = "
<< h_C[0] << std::endl;
    } else {
        std::cout << "Vector addition failed!" << std::endl;
```

```
    }

    // 释放内存
    delete[] h_A;
    delete[] h_B;
    delete[] h_C;
    CUDA_CHECK(cudaFree(d_A));
    CUDA_CHECK(cudaFree(d_B));
    CUDA_CHECK(cudaFree(d_C));

    return 0;
}
```

内存类型	特点	大小限制
栈内存	用于存储局部变量（比如函数里的 `int a=10`），由系统自动分配 / 释放	非常小（默认通常只有**几 MB**，比如 Windows 默认 1MB、Linux 默认 8MB）
堆内存	用于存储动态分配的数据（比如 `new` 出来的数组），由程序员手动分配 / 释放（`delete`）	非常大（通常是系统的可用物理内存，以**GB 为单位**）

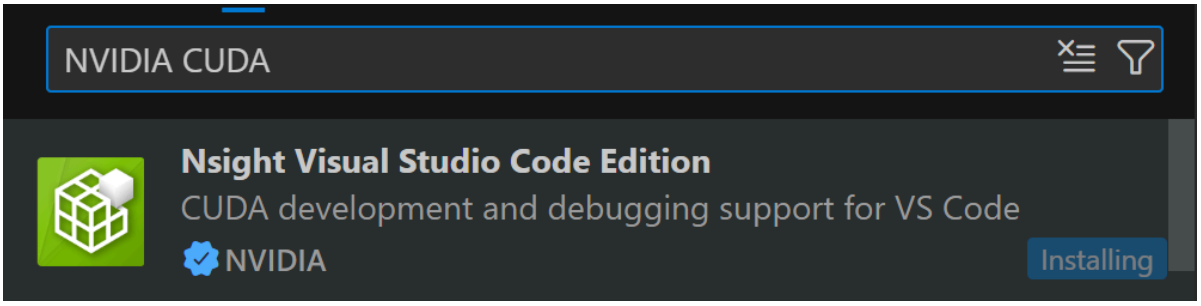
CUDA Toolkit 下载与安装

问题：vs版本太高导致的，**先继续安装 CUDA，后续手动配置 VS 集成**：勾选安装界面的“`I understand...`”复选框完成 CUDA 安装，之后需要手动在 VS 中配置“CUDA 编译工具（nvcc）”的路径，才能编译 CUDA 代码。

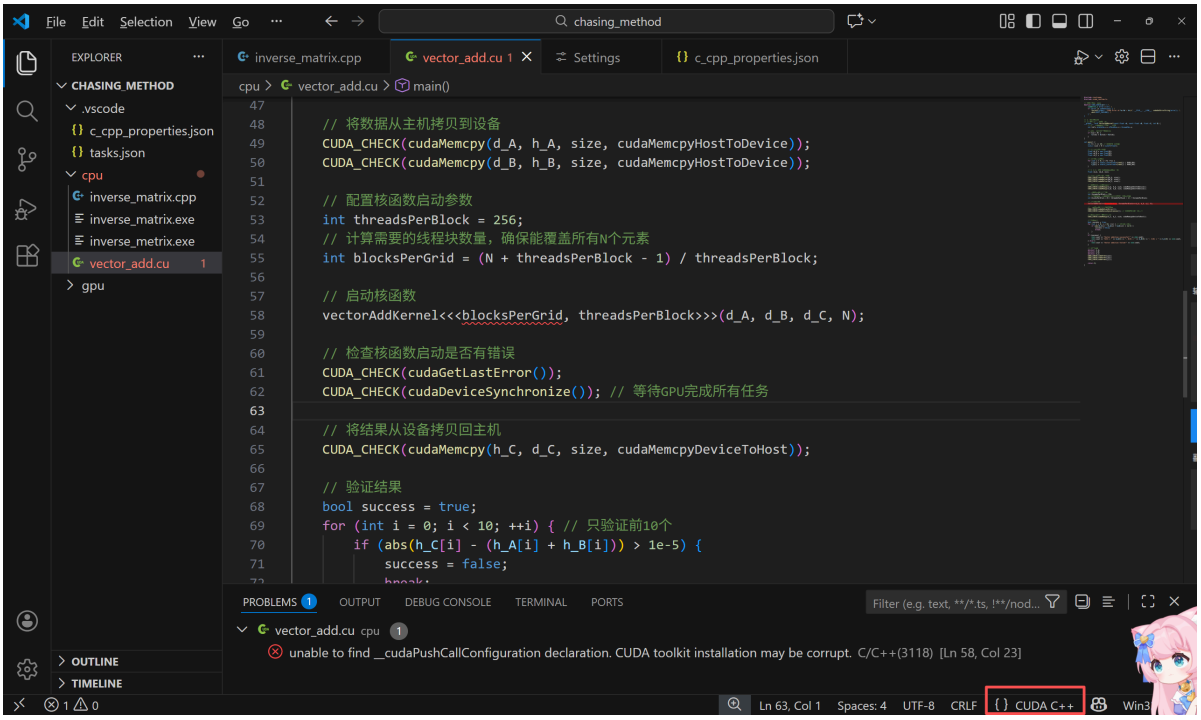


保存文件后，VSCode 会自动刷新 IntelliSense，`#include <cuda_runtime.h>` 的红色波浪线会消失。

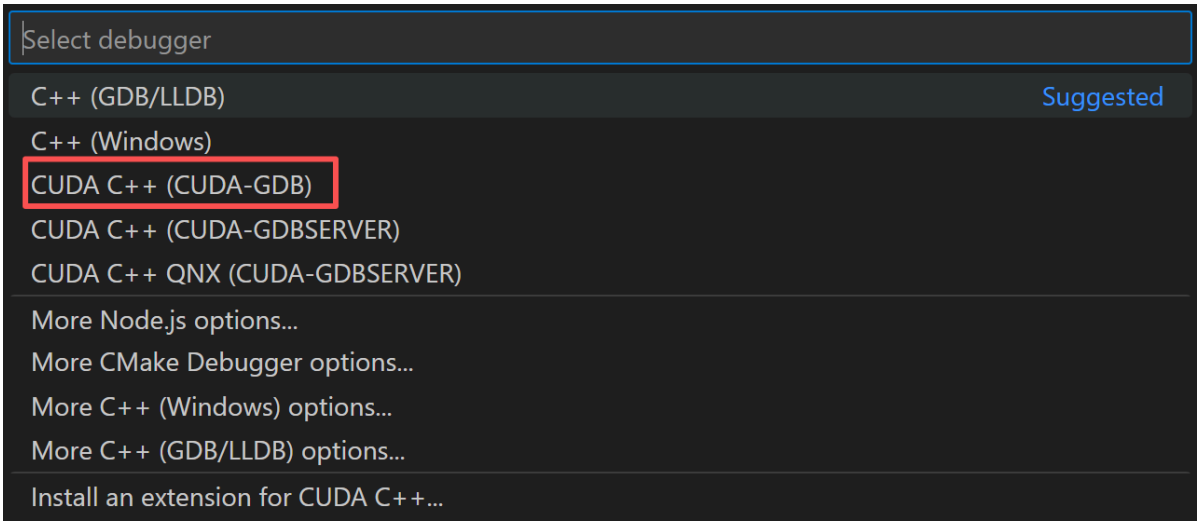
vscode中增装一个拓展，用于cuda编程：



右下角红框的位置是“语言模式”！



.cu文件选择编译器：cuda c++



vscode 删除一整行的快捷键：ctrl+shift+K

Windows 命令行默认是 `GBK` 编码，而 CUDA 程序编译后默认输出 `UTF-8`，所以会乱码。先在命令行里切换到 UTF-8 编码：

在运行程序前，先输入：

```cmd
chcp 65001
```

chcp 65001

GPU（图形处理器，Graphics Processing Unit）本质：硬件设备，是显卡的核心芯片

CUDA（Compute Unified Device Architecture）本质：NVIDIA 推出的

并行计算平台和编程模型（纯软件范畴），不是硬件，而是一套完整的工具链，包括：

- 编程接口（类 C/C++ 的 CUDA C/C++）；
- 编译器（nvcc）、运行时库；
- 高性能算法库（如 cuBLAS、cuDNN、cuFFT，用于 AI、数值计算）；
- 调试 / 性能分析工具（如 Nsight）。

nvcc（CUDA 专用编译器）的隐式引入cuda_runtime.h，`nvcc` 是 NVIDIA 为 CUDA 定制的编译器，它会对代码做特殊预处理。所以，即使没有显式的写#include <cuda_runtime.h>但是依然能够正常运行，此写法不推荐。

```
//cuda错误检查宏，gpu调试核心//copy//failure
#define CHECK(call)\
    do{\
        cudaError_t err=call;\
        if(err!=cudaSuccess){\
            printf("cuda错误%s行号: %d信息: %s\n",_FILE_,_LINE_,cudaGetErrorString(err));\
            exit(EXIT_FAILURE);\
        }\
    }while(0)
```

宏定义（纯文本替换）里不能轻易地加注释！会报错

一次核函数调用（通过 `<<<grid维度, block维度>>>` 语法启动），必然对应且仅对应一个线程格（grid）。核函数调用次数 = 线程格数量

核函数没有返回值：void，上百万个线程会有上百万个返回值，这无意义！

编译器会把 `*` 与变量名绑定

- 若写 `float *a, *b;`：清晰表示 `a` 和 `b` 都是 `float` 类型的指针；
- 若写 `float* a, b;`：容易被误解为“`a` 和 `b` 都是指针”，但实际 `b` 是普通 `float` 变量（因为 `*` 只绑定到 `a`）。

`cudaMalloc` 的定义是：cudaError_t cudaMalloc(void **devPtr, size_t size);

它的第一个参数是**二级指针（`void\**`）**—— 作用是：把 “GPU 显存分配后的地址”，写入到「主机端的指针变量」里。`&d_a` 就是主机端指针 `d_a` 的地址

。

`d_a`：指针变量本身；`*d_a`：解引用 → 访问 “d_a 指向的 GPU 显存空间”；`&d_a`：取地址 → 存储 “d_a 这个变量在主机内存的地址”。

Synchronize同步

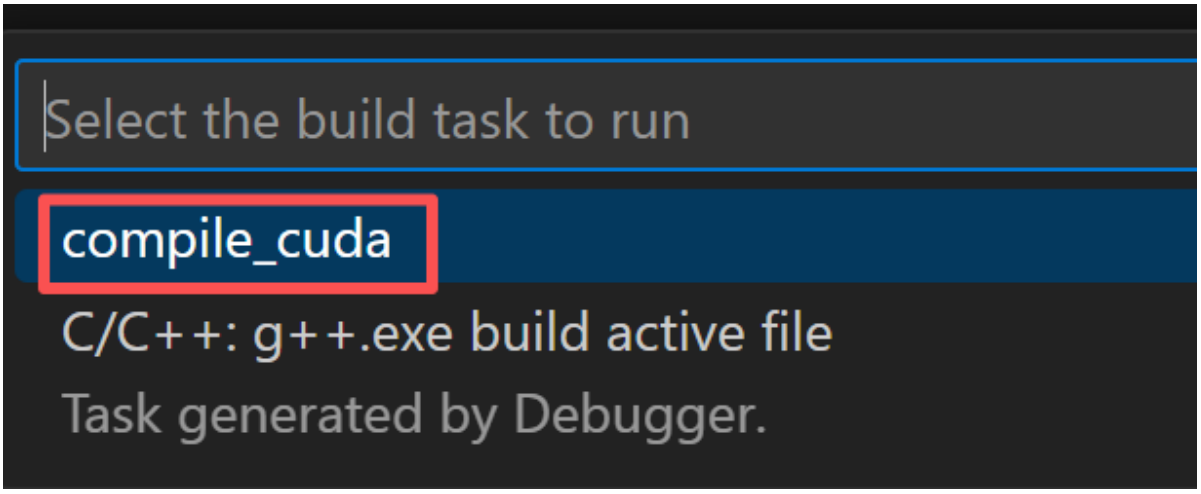
float *h_a = new float[n];

`new` 是动态内存分配运算符，在主机端的**堆内存**中，分配一块连续的、能容纳 `n` 个 `float` 元素的内存空间。`h_a` 是一个 `float*` 类型的指针变量，它的作用是存储 `new float[n]` 返回的 “数组起始地址”——此时 `h_a` 就相当于这个动态数组的 “首地址指针”。动态分配的堆内存不会自动回收，使用完后要通过 `delete[] h_a;` 释放（否则会导致内存泄漏）。

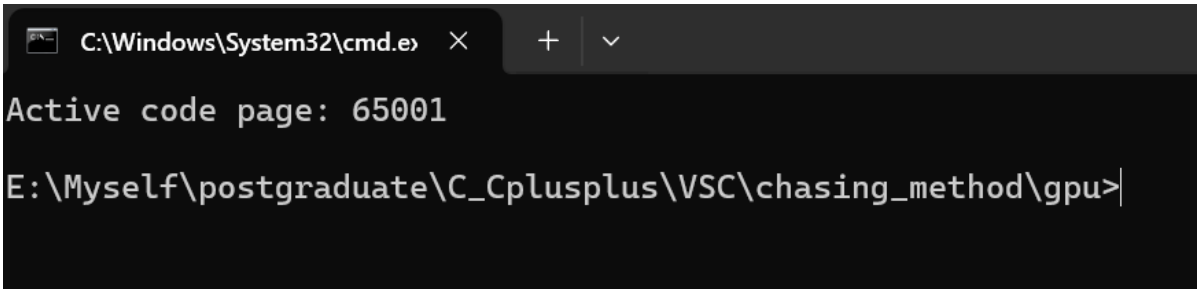
`cudaFree` 只释放 “手动分配的 GPU 显存”；`cudaDeviceReset()` 释放的是 “进程在 GPU 上的所有资源”，兜底清理。

我的vscode，.cu文件的编译运行流程：

编译：ctrl+shilf+b，选择compile_cuda(它是在配置文件.json中设置的------**回深入了解**)



到指定路径下的终端运行，chcp 650001，先记得改一下编码。



推进隐式计算：12月17日

> 你老师建议用**一维热传导方程**理解隐式算法的思路，和我之前回答的核心逻辑是**完全适配的**（因为一维 Couette 流动的控制方程和一维热传导方程都属于**抛物型偏微分方程**，隐式算法的 “离散→方程组→求解” 套路完全一致），只是案例场景更简单、更易入门。我帮你把学习步骤调整为**以一维热传导方程为核心案例**，更贴合老师的建议：

老师建议和课本的差异！

主对角线diagonal

递推recurrence

前缀prefix

记号notation

尾数mantissa

系数coefficients

先明确：Python 中 “一切皆对象”

Python 里所有数据都是对象，分为两类：

- **不可变对象**：int、str、tuple、float 等（无法原地修改内容）；
- **可变对象**：list、dict、numpy 数组、自定义类实例等（可以原地修改内容）。

对比python与C++的函数参数传递。

文档中最终的非定常控制方程是8.18，它是**纳维 - 斯托克斯方程（NS 方程）** 经过 4 个关键假设简化而来，每个假设都有明确的物理意义，缺一不可：

假设条件	物理意义	对方程的影响
1. 不可压缩流动	流体密度rho为常数（低速流动满足）	连续方程简化，密度可从导数中提出
2. x方向无穷远	平板沿x方向无限长，流动参数（速度u、压力p）与x无关	(\frac{\partial}{\partial x}=0)，所有沿x的偏导数都消失
3. 垂直方向速度(v=0)	流体只有水平方向（x向）速度，没有上下（y向）运动	动量方程中含v的项全部为 0
4. 非定常（保留时间导数）	流动随时间变化（从初始状态过渡到稳态）	保留(\frac{\partial u}{\partial t})项，方程从 “定常” 变为 “非定常”

物理意义	对方程的影响
流体密度ρ为常数（低速流动满足）	连续方程简化，密度可从导数中提出
平板沿x方向无限长，流动参数（速度u、压力p）与x无关	$\frac{\partial}{\partial x}=0$，所有沿$x$的偏导数都消失
流体只有水平方向（x向）速度，没有上下（y向）运动	动量方程中含v的项全部为 0
流动随时间变化（从初始状态过渡到稳态）	保留$\frac{\partial u}{\partial t}$项，方程从 “定常” 变为 “非定常”

简化结果：复杂的 NS 方程最终变成一个**抛物型偏微分方程（PDE）** —— 可以理解为 “流体的黏性（右边μ项）是‘扩散源’，将上板的运动（剪切力）沿y方向扩散，时间（左边$\frac{\partial u}{\partial t}$）是扩散的过程”。

隐式方法：Crank-Nicolson 格式是什么？（为什么要用隐式？）

文档用**Crank-Nicolson 隐式有限差分方法**求解式 (8.20)，核心是解决 “显式方法的时间步限制”—— 显式方法受 CFL 准则约束，Delta t必须很小，否则不稳定；而隐式方法**无条件稳定**，可以用大Delta t，大幅减少计算步数。

1. **隐式格式的核心逻辑（不用深究公式推导，理解 “隐式” 的含义）**

- 显式：用 “第n时刻的已知速度” 直接计算 “第$n+1$时刻的速度”（比如$u_j^{n+1}=f(u_j^n,u_{j\pm1}^n)$）；
- 隐式：第$n+1$时刻的速度（未知量）会出现在方程两边（比如$Au_{j-1}^{n+1}+Bu_j^{n+1}+Au_{j+1}^{n+1}=K_j$），需要解一个方程组才能得到$u_j^{n+1}$。

2. **方程组的特点：三对角矩阵**

由于每个网格点j的速度只和相邻的$j-1$、$j+1$点有关，方程组的系数矩阵是 “三对角矩阵”（只有主对角线和上下两条副对角线非零），用**Thomas 方法（追赶法）** 可以高效求解，避免直接求逆的复杂计算。

核心逻辑链（一句话串起来）

两平行平板间的非定常黏性流动 → 简化 NS 方程得到抛物型 PDE → 无量纲化引入雷诺数 → 用无条件稳定的 Crank-Nicolson 隐式方法离散 → 三对角方程组用 Thomas 方法求解 → 选择合适的 E 参数平衡计算效率和精度 → 最终收敛到线性稳态解（与精确解一致）。

1. **先明确：什么是 “抛物型偏微分方程”？**

你现在的控制方程（8.20）是**黏性扩散类方程**（流体速度随时间的变化由黏性 “扩散” 决定），这类方程和 “热传导方程”（热量从高温区扩散到低温区）属于同一类，数学上叫 “抛物型偏微分方程”。

这类方程的特点是：**“扩散过程” 有明确的传递速度**—— 比如热量不会瞬间传遍整个区域，黏性也不会瞬间把上板的速度传到下板。

`np.asarray` 是 NumPy 库的核心函数之一，核心功能是将输入的非 NumPy 数组类型（或已有 NumPy 数组）转换为 NumPy 数组（ndarray 对象）。

`np.zeros_like` 是 NumPy 库的常用函数，核心功能是创建一个与输入对象 `y` “形状完全一致” 且 “元素全为 0” 的 NumPy 数组（ndarray）。

要记得再理解理解解析解！！！！！！！！！

- **GPU** = 千军万马
- **CUDA** = 让你能指挥这千军万马的“军令系统”

基于 CUDA 加速的一维 Couette 流（库埃特流）数值求解器

维度	速度场形式	空间梯度	物理场景	离散后方程组
一维	u = u(y, t)，v=0	仅 y 方向梯度	无限大平行平板纯剪切流	一维三对角方程组（托马斯算法可解）
二维	u=u(x,y,t)，v=v(x,y,t)	x、y 方向均有梯度	有限宽平板、带压力梯度的库埃特流	二维块三对角 / 五对角方程组（需隐式格式 + 迭代 / 分块求解）

主流方向（x）的速度分量 `u` 与垂直主流方向的速度分量 `v`。

`cudaError_t` —— 错误码类型（枚举类型）

- 它是 C/C++ 标准库中预定义的三个标准流之一：
  - `stdin`：标准输入（键盘）
  - `stdout`：标准输出（屏幕，通常用于正常输出）
  - `stderr`：标准错误输出（屏幕，但专门用于错误信息）

托马斯算法前向后向替换使用的是串行递推，无法实现并行逻辑，此时，应该放到cpu端执行。因为，如果花费一个gpu线程串行执行反而没有cpu单核效率高，还要承担启动核函数、数据拷贝等开销。

std::vector v = {1.0, 2.0, 3.0}; std::vector* lower = &v;

指向一个 `std::vector<double>` 对象的指针

double arr[3] = {1.0, 2.0, 3.0}; double* lower = arr; // 或 &arr[0]

指向一个 `double` 类型变量（或数组）的指针

cuda:

核函数内部可包含循环（如 m 的循环），循环是每个线程独立执行的，不影响并行性。

标量参数（`t_star`、`ReD`）会自动从 CPU 拷贝到 GPU，供所有线程访问。当你调用 `kernel<<<...>>>` 时，CUDA 会：将 `t_star` 和 `ReD` 的值复制一份到 GPU 的 每个线程的寄存器中，只针对标量参数。指针/数组（double* y）不自动拷贝。

`h_u.data()` 返回的是底层连续数组的首地址（`double\*`），这才是 `cudaMemcpy` 能用的。

ql phd论文

用 CFD 模拟飞机的**定常可压缩流动**时，为了处理复杂几何（比如翼身组合体），先把流场拆成**多块网格**；用**牛顿迭代**解流场方程，但初始解太差，就用**同伦延拓**（伪时间 + 拉普拉斯）帮牛顿 “稳起步”；又因为多块网格容易 “对不齐”，就用**全局求解器**让各块实时同步信息 —— 最终实现 “又快又准的流场计算”，帮飞机气动设计提速。

对比点	不可压缩流动（如水）	可压缩流动（如高速空气）
核心特点	密度不变	密度随压力/速度变
计算难点1（刚性）	压力变化“瞬间传”，常规方法跟不上	压力变化“慢慢传”，能和流速协调
计算难点2（方程）	要满足LBB（速度-压力配对），压力当“砝码”	不用LBB，要同时算密度、能量、激波
典型计算技术	投影法、算子分裂（适配“瞬间传信”）	激波捕捉、高阶格式（稳住“冲击”）

控制方程 = 气流流动的物理规则；

控制方程的解 = 符合这些规则的 “真实气流状态”（每处的速度、压力等参数）；

双曲型方程的解是鞍点 = 这个真实气流状态是 “不稳定的平衡”，计算时容易跑偏，所以需要后续的稳定化处理。

一、 角色定位：我到底在干什么？

你目前的工作处于**计算机科学（CS）与计算流体力学（CFD）的交叉地带。**

- **不是纯粹的CFD物理研究：** 乔老师并没有要求你从物理机理上研究湍流是如何形成的，那是流体力学专业学生的活。
- **不是纯粹的业务层软件开发：** 你不是在写CRUD（增删改查），也不是做移动端APP。
- **你的真实身份是：高性能计算（HPC）工程师 / 科学计算软件专家。**

一句话总结： 你是利用软件工程的专业知识（数据结构、并行计算、工程化思维），为复杂的物理方程（CFD）打造“最快的跑车底盘”（求解器框架）和“最强的引擎”（显卡加速算法）。

二、 如何起步？——“三步走”战略

乔老师让你先看他的博士论文是非常英明的建议。他的论文（）本质上是在解决一个问题：**如何让流场计算收敛得更快、更稳。**

1. 理论基础：先看“算法逻辑”，再看“物理机理”

- **重点阅读导师论文的第三、五、六章：** 重点理解**牛顿迭代法**（）、**GMRES算法**（）和**多块网格耦合**（）。

- **理解线性代数的工程应用：** 软件工程学生对矩阵不陌生，但在CFD里，你会面对数千万维度的稀疏矩阵。理解什么是**雅可比矩阵（Jacobian）**（）以及如何利用**预处理（Preconditioning）**（）提高效率。
- **补一点点CFD基础：** 了解什么是**有限体积法（FVM）**（），知道流场是怎么离散成一个个小格子的。

2. 代码研究：CFL3D vs. 风雷 (PHengLEI)

- **CFL3D（算法之源）：** 它是NASA开发的经典软件（）。乔老师让你看它的核心函数（af3f.F等），是为了让你理解**隐式算子**（）在代码里是怎么实现的。那是前人的智慧结晶，虽然是Fortran写的，但数学逻辑是通用的。
- **风雷（框架之基）：** 这是国内开源的主流CFD框架。作为软件工程背景的你，要发挥优势，研究它的**类设计、内存分配、多块网格的通信逻辑（MPI）**（）。

3. 技术突破：GPU 加速（你的核心价值）

- **CUDA 编程：** 既然目标是显卡，CUDA是绕不开的。乔老师在消息里提到“GMRES算法只涉及矩阵乘向量和向量加减，非常适合并行”（），这正是你的切入点。
- **性能分析：** 学习使用 `nvprof` 或 `Nsight` 分析代码瓶颈，看看计算量到底卡在哪里。

三、 就业前景：这条路值不值得走？

非常值得，而且壁垒极高。

1. **工业软件（CAE/CFD）：** 国家现在极其重视“卡脖子”技术。像中航工业（AVIC）、中国商飞（COMAC）以及国内新兴的工业软件公司，非常渴求既懂数值算法又懂底层优化的人才。
2. **高性能计算（HPC）：** 像 NVIDIA、华为、阿里、腾讯的计算平台部门，需要人去优化大规模并行计算的效率。
3. **稀缺性：** 软件工程的人大多去搞互联网了，流体力学的人大多计算机底层能力稍弱。像你这样能把**复杂数学算法实现在显卡上**的复合型人才，是市场上的“稀有物种”。

四、 给你的具体建议（第一步做什么）

不要试图先去读厚厚的流体力学课本。

1. **第一周：** 读完乔老师论文的**摘要**和**第一章绪论**（），理解他为什么要搞“拉普拉斯延拓”和“全局耦合线性求解”。
2. **第二周：** 配置好 VS Code，把乔老师给你的 `af3f.F`（）代码跑通（哪怕只是编译通过），对比论文里的公式 2-3（对流通量），看看公式是怎么变成代码的。
3. **第三周：** 找一个简单的 CUDA 向量加法例子运行一下。

乔老师非常看重**“颗粒度分配”**和**“并行计算”**（），这说明他希望你从架构层面思考问题。

你想从乔老师论文中哪个具体算法（比如 GMRES 还是多块网格通信）开始更深入地拆解代码实现逻辑吗？

任务一：深度攻克 GMRES 算法与并行逻辑

乔老师特别强调了 **GMRES 算法**非常适合并行且适合上显卡，这是你未来工作的核心。

1. **研读论文第五章**：重点阅读 **5.4 节（基于 GMRES 方法的多块网格全局求解）** 2。
2. **拆解伪代码 5.1**：仔细分析论文中提供的 **“右预处理的 restart GMRES(m) 算法”伪代码** 3。
   - 作为软件工程学生，请将该算法逻辑抽象为：哪些步骤是**向量内积**？哪些是**矩阵-向量乘法**？哪些是**施密特正交化**？
3. **动手实践（代码级）**：不要只看，尝试用 C++ 或 Python（结合 NumPy）自己写一个最简单的 GMRES 求解器，解一个 100×100 的随机稀疏矩阵。这会让你对“计算开销”有直观感受。

任务二：调研“风雷 (PHengLEI)”框架与并行颗粒度

导师提到了**“整个计算框架颗粒度分配”**的问题 4，这直接关系到软件架构。

1. **框架调研**：虽然不用编译 CFL3D，但你需要调研**风雷 (PHengLEI)** 框架的开源文档或相关论文。重点看它的**多块网格数据结构**是如何组织的 5555。
2. **理解“颗粒度”**：
   - 阅读论文 **5.2 节（多块网格的非耦合求解）** 6和 **5.4.5 节（多块分布式求解器的实现）** 7。
   - 思考：如果一个流场有 1000 万个网格，分成 100 个块（Block），每个块由一个 GPU 核心处理，还是每个网格点由一个线程处理？这就是乔老师说的“颗粒度分配” 8。
3. **拆解算法 5.2**：阅读**“非阻塞通信的分布式矩阵乘向量”**算法 9。分析在 GPU 上，如何利用显存交换来处理这些块边界的数据。

任务三：从算法向 GPU 架构映射（理论推演）

1. **数据依赖性分析**：回顾你做过的 CUDA 向量加法。向量加法是“并行无关”的，但 CFD 里的**对称高斯-赛德尔（SGS）预处理** 10 是有前后依赖的。
2. **思考题**：阅读论文 **5.4.3 节（预处理方法）** 11。这种具有前后依赖的算法，如何通过“染色法”或者“分块”在 GPU 上实现并行？（这是你可以拿去和导师深度交流的问题）。
3. **显存预算分析**：阅读论文 **5.4.4 节（全局耦合解法的代价）** 12。乔老师计算了一个 930 万网格的算例需要约 31.2GB 显存。请你尝试推算：如果使用目前主流的 A100 (80GB) 显卡，最大能跑多少万网格的算例？

任务四：准备与导师的“深度对谈”清单

为了体现你这一周的工作量，建议你在本周结束前整理一份文档，包含以下内容：

- **流程图**：将博论里的 GMRES 算法流程图画出来，并标出哪些部分你认为最适合 GPU 加速。
- **架构理解**：谈谈你对“风雷”框架中多块网格耦合通信逻辑的理解 13。
- **具体疑问**：针对 **PHengLEI 已有的 GPU 代码与“结构网格体系”不追求的问题**，准备 1-2 个具体的技术问题询问导师。

总结：本周你的重心

这一周不要再把自己当成一个“读者”，要当成一个**“系统架构师”**。你的目标是：**搞清楚算法里每一行数学公式，在计算机内存里是怎么流动的。**

本周总结 + 疑问：

第一遍先注重理解，第二遍再注重对照实际现象的理解。

比如：

> 各自对应的特征速度分别为 $\boldsymbol{\upsilon} = (u_n + a, u_n - a, u_n)$，其中 u_n 为边界处的外法向速度分量，a 为当地声速。根据边界内侧和外侧的流场状态参数 $\mathbf{w}_i$ 和 $\mathbf{w}_o$，可以分别得到相应的 $\boldsymbol{\varphi}_i$ 和 $\boldsymbol{\varphi}_o$。然后，根据特征速度确定在最终在边界上所施加的约束数值，
>
> $$\varphi_{b,j} = \begin{cases} \varphi_{i,j}, & \upsilon_j > 0 \\ \varphi_{o,j}, & \upsilon_j \leq 0 \end{cases}, \ j = 1, 2, 3. \tag{2-33}$$
>
> 得到 $\boldsymbol{\varphi}_b$ 后，只需将变量还原为控制方程的自变量得到 $\mathbf{w}_b$，然后计算相应的残差即可。
>
> 在实际应用中，各式各样的边界条件由于物理意义的不同，所指定的物理量，即 $\boldsymbol{\varphi}_o$ 的情况有所差异，但最终的施加给控制方程的途径都是符合上述模式的。换一种说法，变量 $\boldsymbol{\varphi}_o$ 中包含的是“想要”施加给流场的边界条件，而至于 $\boldsymbol{\varphi}_o$ 是否能确实施加给流场作为约束，取决于边界处特征速度 $\boldsymbol{\upsilon}$ 的状态，其选择过程是通过公式(2-33)实现的。由于在求解前无法或不便确定边界处的状态，公式(2-33)正是避免施加不适定约束的关键机制。

第一遍：这一段描述了对边界条件约束的筛选机制。第二遍解决保留问题：**为什么特征速度与0做比较能够判断该约束的去留？**

> 在牛顿迭代的每一步中，都需要求解一个线性问题得到解的增量。因此，在引入延拓算子后，除考虑它们对非线性迭代产生影响外，还要分析他们对相应线性算子性质的影响。
>
> 为便于讨论分析，参考 Hicken 和 Zingg 等人的做法[129]，此处以一维定常输运方程为模型问题，讨论不同延拓方法对线性算子的条件数和谱特性影响。

我的理解：

非线性迭代：外层的while循环；

线性问题：该轮次处理的核心问题。

理解上有些抽象，老师，您论文中出现了很多线性非线性，这一块说法上，我理解的不太到位。

```
// 1. 非线性迭代 (外层 while 循环)
while (residual > 1e-11) {
    // 2. 构造线性算子 (构建雅可比矩阵 A)
    // 论文的核心点：在这里引入“拉普拉斯延拓”来修改 A 的性质
    LinearOperator A = Build_Linearized_Operator(current_w);
    Vector b = Calculate_Current_Residual(current_w);

    // 3. 求解线性问题 (这是你的核心任务：用 GPU/GMRES 算 delta_w)
```

```
    // 线性算子的性质决定了这一步的速度
    Vector delta_w = GMRES_Solver(A, b);

    // 4. 更新状态 (得到解的增量)
    current_w = current_w + delta_w;

    residual = b.norm(); // 更新残差，继续下一次迭代
}
```

关注点	伪时间项 (PT) 的后果	拉普拉斯项 (LP) 的后果
显存布局	只改对角线，存储极简单。	会增加非零元素的数量（由 3 对角变 5 对角或更多）。
计算模式	局部计算，Thread 之间无依赖。	**扩散操作**。在 GPU 上需要更多的 **Stencil（模板）访问**。
GMRES 压力	条件数好，但可能原地踏步。	谱特性好，GMRES 迭代次数会明显减少。

当看到 $\|R_k^n\| < 1.0^{-11}$ 时，大脑应该自动翻译成下面这段 C++ / CUDA 伪代码：

```
// 执行外层延拓循环 (第 n 步)
for (int n = 0; n < max_continuation_steps; n++) {
    // 执行内层牛顿迭代循环 (第 k 步)
    for (int k = 0; k < max_newton_steps; k++) {
        // 1. 计算当前所有网格点的残差（得到 R 向量数组）
        Vector R = calculate_residual(current_w);
        // 2. 将数组 R 归约为一个总分（计算范数）
        // 在 GPU 上这通常涉及一个高效的 Reduce Kernel
        double total_error_score = R.norm();
        // 3. 检查是否达到最终精度要求
        if (is_final_stage(n) && total_error_score < 1e-11) {
            std::cout << "Debug: 系统已完全收敛，停止计算！" << std::endl;
            return; // 终止程序
        }
        // 4. 如果没达标，继续用 GMRES 算补丁，更新 current_w
        update_solution(current_w);
    }
}
```

理解上的疑问：为什么先外层延拓循环 (第 n 步) 后内层牛顿迭代循环 (第 k 步)?

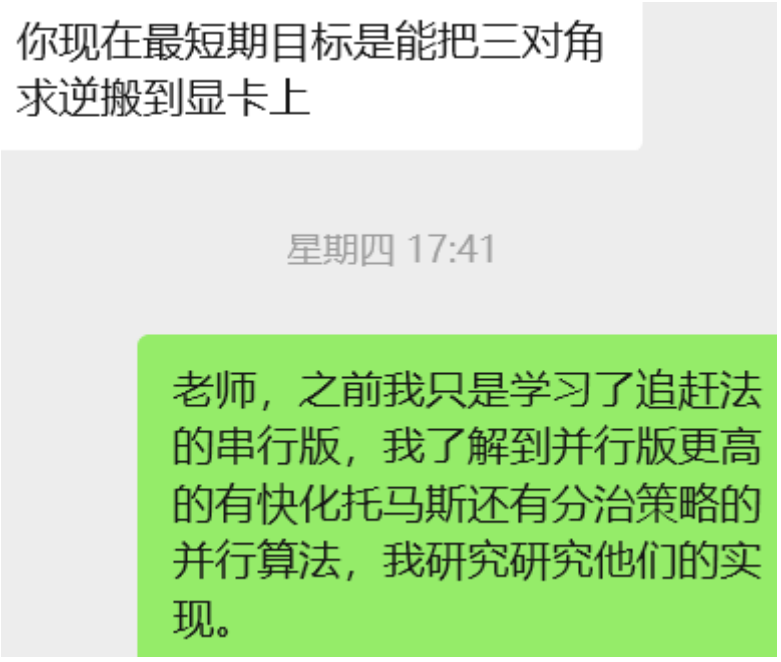
您的论文中选择了弱形式来处理悬点（悬点出现的占比如何？），会出现if-else分支逻辑，这会把整个warp的速度拉低到最慢{warp1-32}，了解到的就是分支发散。

我的思考：是否应该在数据预处理阶段，就根据'网格类型'让执行相同逻辑的网格尽量排在同一个warp里?

带来问题：先计算普通网格，再处理悬点处，会导致该轮次计算结果出错吗?

```
if (这个网格面是普通的) {
    执行标准 Roe 格式计算;
} else if (这个面是悬点 hanging point) {
    执行特殊的插值逻辑和通量交换;
} else if (这个面是物理边界) {
    执行特征边界条件逻辑;
}
```

嘿，官方库了解一下！！！



4.1.3 线性求解器的选择和实现

每个延拓问题仍然是非线性问题，使用牛顿法求解。在每一步牛顿迭代中，都需要求解一个大规模线性问题。线性问题的求解方法分为迭代法和直接法两类。考虑到本章测试问题的规模较小，采用直接求解器的计算代价在可以接受的范围内。而迭代法中的雅可比迭代效率较低，高斯-赛德尔迭代类的算法依赖矩阵的行列排序，以 GMRES 为代表的 Krylov 子空间方法的效率又严重依赖预处理方法，为避免使问题不必要地复杂化，同时排除迭代求解器的效率和精度对非线性求解方法效率对比的污染，本章采用基于并行的 LU 分解方法的线性求解器。在程序中，雅可比矩阵的计算通过自动微分 Sacado 精确实现，雅可比矩阵和残差向量的数据结构通过 PETSc[155] 库实现，然后通过 MUMPS[176,177] 提供的多波前（multi-frontal）并行直接求解器求解。从有限元离散到求解器数据结构的对接是通过有限元库 deal.II[178] 提供的接口实现的。

“局部贡献” 指的是单个网格单元对全局方程组（*KU=F*）的单元级贡献，也就是局部刚度矩阵 `local_K` 和局部载荷向量 `local_F`。

悬点与普通节点的积分项是 “独立可分” 的

在单元的积分计算中，悬点和普通节点对应的积分项是**相互独立**的 —— 悬点的积分项只依赖其插值后的形函数和自身的积分域子区域，普通节点的积分项只依赖自身的形函数和积分域子区域，二者之间没有 “计算依赖”：

- 普通节点的积分项：∫Ω∇N普通·∇N普通dΩ
- 悬点的积分项：∫Ω∇N悬点（插值）·∇N普通dΩ

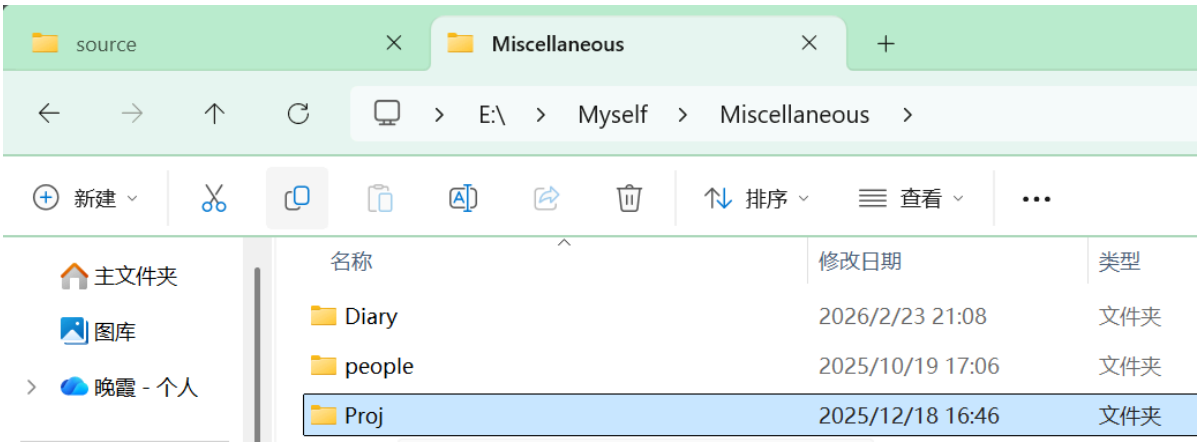
这两个积分项是“并行的数学运算”，先算哪一个，都不会改变各自的积分结果，最终合并后的局部贡献值也完全相同。

组装矩阵的过程是不用考虑顺序（普通节点与悬点的顺序）的，在矩阵组装之前，计算当前单元局部贡献的时候，该过程的积分运算与顺序也是无关的。

```
def main():
    # 步骤1：初始化网格（示例：包含悬点的简单网格）
    # 创建常规节点
    node1 = Node(node_id=1, is_hanging=False)
    node2 = Node(node_id=2, is_hanging=False)
    # 创建悬点（父节点为node1、node2）
    node3 = Node(node_id=3, is_hanging=True, parent_nodes=[node1, node2])
    # 创建两个单元：单元1（普通节点）、单元2（包含悬点）
    cell1 = Cell(cell_id=1, nodes=[node1, node2])
    cell2 = Cell(cell_id=2, nodes=[node2, node3])
    all_cells = [cell1, cell2]

    # 步骤2：弱形式参数（可根据实际方程调整）
    weak_form_params = {
        "stiffness_coeff": 0.1,     # 刚度系数（弱形式积分结果）
        "load_coeff": 0.5,          # 载荷系数
        "interp_weights": [0.6, 0.4]  # 悬点插值权重
    }

    # 步骤3：遍历所有单元，执行「局部计算 → 全局求和组装」
    for cell in all_cells:
        # 3.1 计算当前单元的局部贡献（含悬点处理）
        local_K, local_F = compute_local_contribution(cell, weak_form_params)
        # 3.2 将局部贡献累加（求和）到全局矩阵/向量（组装核心）
        assemble_global_matrix(cell, local_K, local_F)

    # 输出结果：全局矩阵/向量是所有单元局部贡献的和
    print("全局刚度矩阵（局部求和结果）：", global_K)
    print("全局载荷向量（局部求和结果）：", global_F)
```

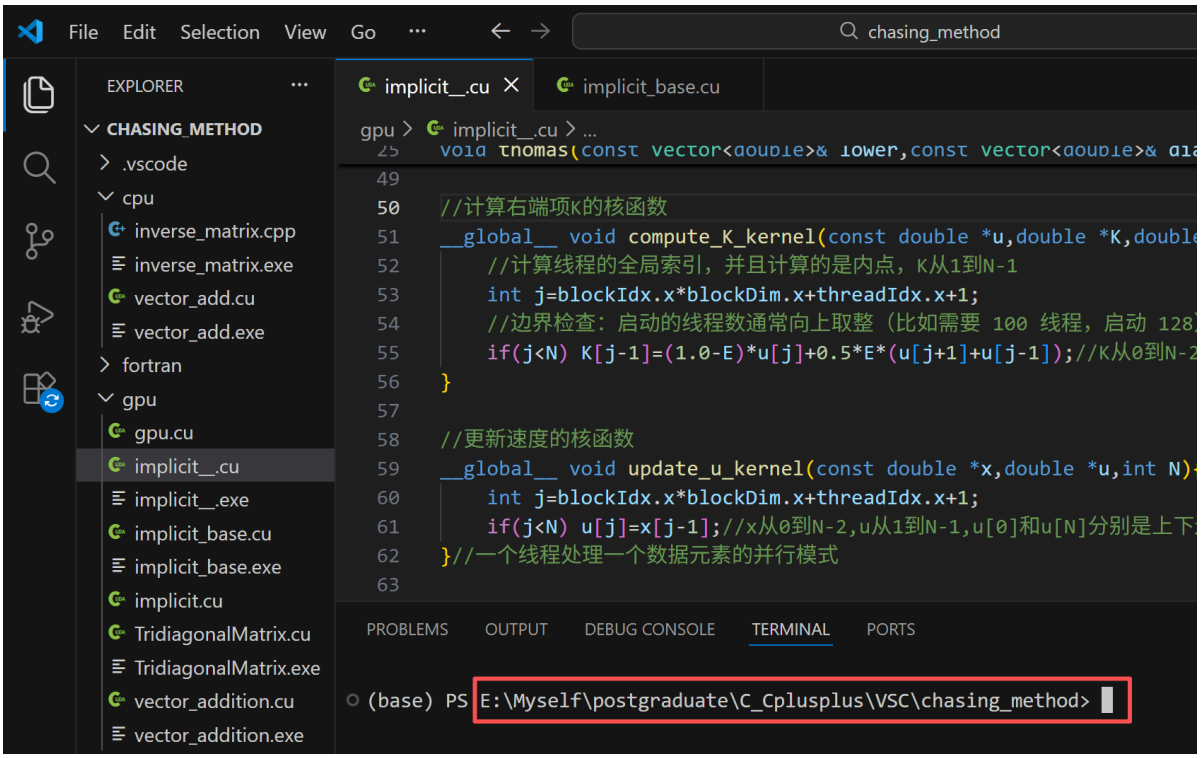
这个局部贡献值就是通过积分来计算的，与顺序也是无关的。
刚刚的是属于：有限元路线，是支线任务！

记录：



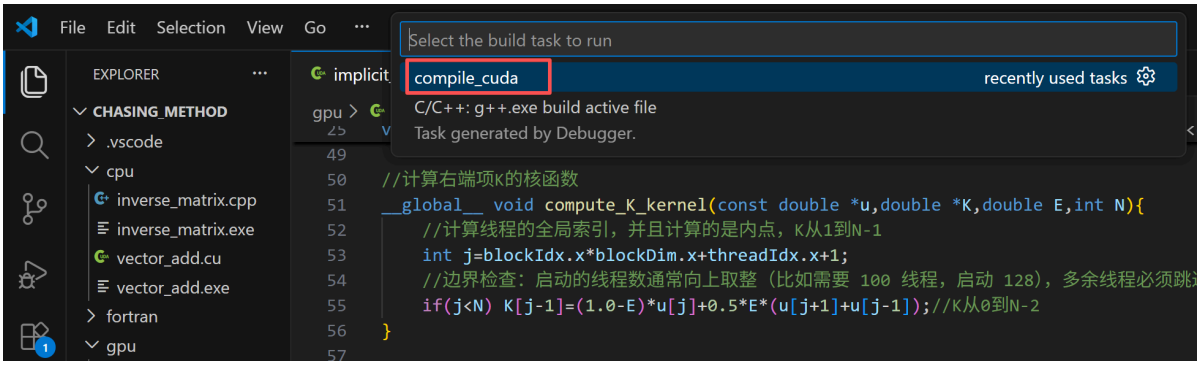
python项目位置

vscode项目位置（需要回顾并行操作）

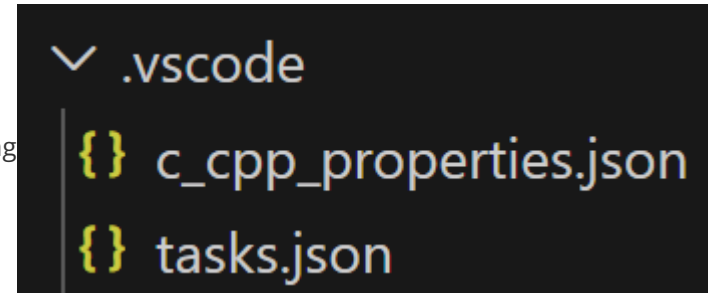


cuda运行

ctrl+shift+b---选第一个（成功编译 **运行再回忆**）



![image-20260304152105209](C:\Users\23608\AppData\Roaming\Typora\typora-user-images\image-20260304152105209.png



文件1:

```
{
    "configurations": [
        {
            "name": "Win32",
            "includePath": [
                "${workspaceFolder}/**",
                "${env:CUDA_PATH}/include" // CUDA头文件路径（核心）

            ],
            "defines": [
                "_DEBUG",
                "UNICODE",
                "_UNICODE"
            ],
            "compilerPath":
"E:\\ProgrammingSoftware\\Env\\C++interpreter\\mingw\\x86_64-15.2.0-release-win32-seh-ucrt-rt_v13-rev0\\mingw64\\bin\\gcc.exe",
            "cStandard": "c17",
            "cppStandard": "gnu++17",
            "intelliSenseMode": "windows-gcc-x64"
        }
    ],
    "version": 4
}
```

文件2:

```
{
    "tasks": [
        {
            "type": "cppbuild",
            "label": "C/C++: g++.exe build active file",
            "command":
"E:\\ProgrammingSoftware\\Env\\C++interpreter\\mingw\\x86_64-15.2.0-release-win32-seh-ucrt-rt_v13-rev0\\mingw64\\bin\\g++.exe",
            "args": [
                "-fdiagnostics-color=always",
                "-g",
                "${file}",
                "-o",
                "${fileDirname}\\${fileBasenameNoExtension}.exe"
            ],
            "options": {
                "cwd": "${fileDirname}"
            },
            "problemMatcher": [
```

```
                "$gcc"
            ],
            "group": {
                "kind": "build",
                "isDefault": true
            },
            "detail": "Task generated by Debugger."
        },
        // 任务2：编译cu文件（用nvcc）
        {
            "label": "compile_cuda",
            "type": "shell",
            "command": "nvcc",
            "args": [
                "${file}",
                "-o", "${fileDirname}/${fileBasenameNoExtension}.exe",
                "-arch=sm_86",
                "-lcudart"
            ],
            "options": {
                "cwd": "${fileDirname}"
            },
            "problemMatcher": ["$nvcc"],
            "group": {
                "kind": "build",
                "isDefault": true
            }
        }
    ],
    "version": "2.0.0"
}
```